

Predictive Parsing Table Complexity Measurement – Explanation Document

RA2311026050273 – PRIYA DHARSHINI [CSE AIML D]

1. Problem Understanding

Predictive parsing tables are used in compiler design to guide syntax analysis. These tables contain production rules for different combinations of non-terminals and terminals. However, when the table contains too many filled entries, it becomes complex and affects parsing efficiency.

This problem focuses on analyzing the parsing table by computing a feature called **Parsing Table Density**. Based on this value, the program determines whether the table is simple or complex.

2. Logic Used

The program is implemented using Python with the Pandas library.

- The parsing table is represented as a 2D list where each cell contains either a production rule or is empty (None).
- The table is converted into a DataFrame for easy processing.
- The total number of cells is calculated using:
$$\text{Total Cells} = \text{Number of Rows} \times \text{Number of Columns}$$
- The number of filled cells is calculated by counting all non-empty entries.
- The parsing table density is calculated as:
$$\text{Density} = \text{Filled Cells} / \text{Total Cells}$$

- A condition is applied:
 - If density is greater than 0.5, the table is considered complex.
- Based on this, a flag **Is_Table_Complex** is generated.

3. Justification of Threshold / Feature

The feature **Parsing Table Density** is used to measure how filled the table is.

- A low density means fewer entries, making the table simple and efficient.
- A high density means many entries, increasing complexity and reducing efficiency.

The threshold is chosen as:

- $\text{Density} > 0.5 \rightarrow \text{Complex Table}$

If more than half of the table is filled, it indicates higher parsing complexity, so the flag is set to identify it as complex.

Conclusion

This program demonstrates how structural data like parsing tables can be analyzed to measure complexity. By computing parsing table density and applying a threshold, it provides a simple way to evaluate parser efficiency and identify complex grammar structures.